UNIVERSITY OF DHAKA

RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing

Submitted by

Class Roll: 50028

Registration No: H-55

Class Roll: 50040

Registration No: H-49

Submitted to the

Department of Computer Science and Engineering

University of Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of

Professional Masters in Information and Cyber Security (PMICS).

Declaration Form

University of Dhaka

The work in this report titled "**RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing**" represents the original work of the undersigned, carried out under the supervision of Dr. Md. Abdur Razzaque. The authors further declare that where information draws on the literature, the original sources have received adequate citation and referencing. The authors did not submit any part of this report earlier for another degree or diploma at this or any other institution.

Name of the Candidate
Nishan Paul

Name of the Candidate
Md Rakibur Rahman

Dr. Md. Abdur Razzaque
Professor and Chair
Department of Computer Science and Engineering
University of Dhaka

"To conquer fear, you must become fear."

Ra's al Ghul, Batman Begins

UNIVERSITY OF DHAKA

Abstract

Autonomous penetration testing agents powered by large language models have shown rapid progress in recent years, yet published evaluations consistently point to two recurring weaknesses: agents explore candidate vulnerabilities too narrowly, and they struggle to reason about the prerequisite relationships between different stages of an attack. These weaknesses limit how far current systems can operate reliably against realistic, multi-step web, API, and network targets without close human supervision.

This report documents the first stage of an investigation into whether explicitly modeling the dependency relationships between attack steps — rather than treating candidate vulnerabilities as an unordered list — can improve how autonomous agents explore and rank their actions during a penetration test. As an initial step, the authors reviewed a body of recent literature on LLM-based offensive security agents, covering single-agent systems, multi-agent orchestration frameworks, planning-augmented approaches, and the benchmarks used to assess them. Two recurrent failure modes — insufficient exploration breadth and weak prerequisite reasoning — surfaced across independently developed systems. These observations motivate the research direction described in this report, which goes by the name RedGrid.

At this stage, the work is preliminary: a structured review of the current state of autonomous penetration testing research and an early architectural direction motivated by the gaps observed in the literature. The remaining work over the coming months focuses on refining this direction, implementing and assessing it against established benchmarks, and determining whether dependency-aware exploration meaningfully improves autonomous exploitation performance.

Acknowledgements

Sincere gratitude goes to the supervisor, Dr. Md. Abdur Razzaque, Professor and Chair, Department of Computer Science and Engineering, University of Dhaka, for agreeing to guide this work and for the time he has already invested in it at such an early stage. His feedback pushed the authors to move beyond simply summarizing existing autonomous penetration testing systems and to instead ask what specific gap in that body of work this research could realistically address. That question shaped the direction this thesis has taken so far, and will continue to shape the work as it progresses over the coming months.

Acknowledgement also goes to the wider research community whose published work — spanning benchmark design, agent architectures, and empirical evaluations in offensive security — provided the foundation on which this thesis rests. Progress over the next six months depends heavily on the groundwork laid by that literature.

Contents

List of Figures

List of Tables

List of Algorithms

Acronym	**W**hat (it) **S**tands **F**or
ADM	**A**ttack **D**ecision-**M**aking
AI	**A**rtificial **I**ntelligence
API	**A**pplication **P**rogramming **I**nterface
CVE	**C**ommon **V**ulnerabilities and **E**xposures
CVSS	**C**ommon **V**ulnerability **S**coring **S**ystem
CTF	**C**apture **T**he **F**lag
DAG	**D**irected **A**cyclic **G**raph
EL	**E**nvironmental **L**ayer
GPT	**G**enerative **P**re-trained **T**ransformer
LLM	**L**arge **L**anguage **M**odel
PDDL	**P**lanning **D**omain **D**efinition **L**anguage
RCE	**R**emote **C**ode **E**xecution
ReAct	**Re**ason + **Act** (agent framework)
REST	**Re**presentational **S**tate **T**ransfer
SQL	**S**tructured **Q**uery **L**anguage
SQLi	**SQL** **I**njection
SSRF	**S**erver-**S**ide **R**equest **F**orgery
SSTI	**S**erver-**S**ide **T**emplate **I**njection
UCB	**U**pper **C**onfidence **B**ound
VAPT	**V**ulnerability **A**ssessment and **P**enetration **T**esting
VDG	**V**ulnerability **D**ependency **G**raph
WAF	**W**eb **A**pplication **F**irewall
XSS	**C**ross-**S**ite **S**cripting

Dedicated to those who asked the harder question.

Chapter 1

Introduction

1.1 Background and Motivation

1.1.1 Cybersecurity and Vulnerability Assessment

Modern organisations depend on networked software systems for nearly every operational function, and the security of those systems has become a central concern. A security vulnerability is a flaw in software, configuration, or design that an attacker can exploit to gain unauthorised access, execute arbitrary code, disrupt service, or exfiltrate data. Vulnerabilities are regularly discovered in web applications, APIs, network services, and operating systems, and the window between discovery and the deployment of a fix can leave a system exposed for days or weeks.

Vulnerability Assessment and Penetration Testing (VAPT) is the practice of proactively identifying and demonstrating these weaknesses before an adversary does. A penetration test is an authorised, systematic attempt to discover and exploit weaknesses in a target system, typically following a structured workflow: initial reconnaissance to understand the target's attack surface, identification of candidate weaknesses, exploitation of confirmed vulnerabilities, and reporting of findings with remediation guidance. VAPT is widely recognized as an essential security practice,

and demand for it has grown considerably as organisations increase their exposure through cloud services, APIs, and web applications.

1.1.2 Why Conventional VAPT Has Limits

Despite its importance, conventional penetration testing faces practical constraints. Skilled human testers are scarce and expensive. A thorough engagement against a moderately complex target can take days of expert effort. Scheduled, periodic testing leaves gaps between assessments during which new vulnerabilities may arise. Automated scanning tools — such as network port scanners and web application vulnerability scanners — can partially fill this gap, but they generally operate on fixed rule sets and signature databases, and they lack the contextual reasoning needed to chain together exploits. They are effective for known, templatable vulnerabilities, but they cannot adapt to novel application logic, reason about prerequisite relationships between attack steps, or operate across the breadth of a modern attack surface without human guidance.

These limitations have motivated a long-standing research interest in automating parts of the VAPT process.

1.1.3 Large Language Models and AI Agents

Large Language Models (LLMs) are deep learning models trained on large corpora of text that have demonstrated a remarkable ability to follow instructions, reason through complex problems, write and execute code, and interact with external tools. Models such as GPT-4 have achieved strong performance across a wide range of tasks that once required domain expertise [3].

An **LLM agent** extends a base language model with the ability to take actions in an environment: calling tools (such as a web browser, a terminal, or an API), observing the results, and iterating toward a goal. The simplest agents follow a *Reason + Act* (ReAct) loop — reason about the current state, decide on an action,

observe the outcome, and repeat. More sophisticated designs introduce hierarchical planning, role-specialised sub-agents, persistent memory, and structured validation.

The emergence of capable LLM agents has opened a genuinely new question for cybersecurity: *can an LLM agent carry out penetration testing tasks autonomously, without step-by-step human direction?*

1.1.4 The Emergence of Autonomous VAPT Research

Over the past two years, a growing body of research has answered parts of this question affirmatively. Early work by Fang et al. [4] showed that a single GPT-4 agent with browser access could autonomously exploit simplified web vulnerabilities. A later study by the same group demonstrated that GPT-4, when given a CVE description, could exploit 87% of a set of 15 real-world one-day vulnerabilities — while every other tested model and open-source scanner achieved 0% [5]. Importantly, without the CVE description, GPT-4's success rate collapsed to 7%, revealing a sharp distinction between the ability to exploit a known vulnerability and the ability to discover one independently.

These results prompted a rapid expansion of the research area. Zhu et al. [6] demonstrated that a hierarchical team of planning and task-specific agents could autonomously exploit real zero-day vulnerabilities — vulnerabilities for which no description is provided. Deng et al. [7] identified a recurring failure mode in single-agent systems: context loss over long sessions, which causes agents to lose track of what they have already tried and repeat unproductive actions. Later work proposed multi-agent architectures, memory-augmented systems, and planning-augmented approaches to address these and related limitations.

A consistent finding across these papers is that **architecture, not model scale, appears to be the dominant variable in autonomous exploitation performance** — a well-structured pipeline running a comparatively inexpensive model

tends to outperform an unstructured, loosely structured agent loop running a frontier model. This finding, observed independently across six systems in the review, shaped the direction of this thesis.

1.1.5 Motivation for This Thesis

The starting point for this work was to understand this body of research well enough to determine whether it had already addressed the core challenges of autonomous VAPT, and if not, where exactly the remaining gaps lie. Over the past few weeks, the authors assembled and studied a focused set of papers, spanning single-agent exploitation studies, hierarchical multi-agent frameworks, planning-augmented approaches, and the principal benchmark suites used to assess autonomous VAPT agents (surveyed in full in Chapter 2).

Two recurring failure modes surfaced across independently developed systems, and they are the gaps this thesis has chosen to investigate first. The intent here is to be upfront that at this stage this is a direction under active development — not a result already obtained.

1.2 Problem Statement

The literature reviewed during the preparation of this thesis points to two recurring failure modes that appear across independently developed autonomous VAPT systems. Quantitative evidence for both comes from dedicated benchmark studies that allow researchers to measure the failures, not merely describe them. The two modes are not unrelated: they form two sides of the same underlying problem, as the discussion following each failure mode shows.

Failure Mode 1 — Insufficient exploration. On CVE-Bench [1], a benchmark of 40 critical (CVSS $\geq$ 9.0) real-world web-application CVEs, the strongest reported

agent exploits only 13% of one-day and 10% of zero-day vulnerabilities. CVE-Bench's own failure-mode breakdown identifies *insufficient exploration* as the dominant cause of failure across every agent and evaluation setting, with exploration-failure rates ranging from 37.5% to 80.0% (T-Agent: 80.0% zero-day / 55.0% one-day; AutoGPT: 72.5% / 45.0%; Cy-Agent: 67.5% / 37.5%), as illustrated in Figure 1.1. Crucially, this does not appear to be a reasoning-quality problem so much as a breadth-of-search problem: agents commit early to a narrow set of candidate attack paths and rarely return to explore the rest of the surface.

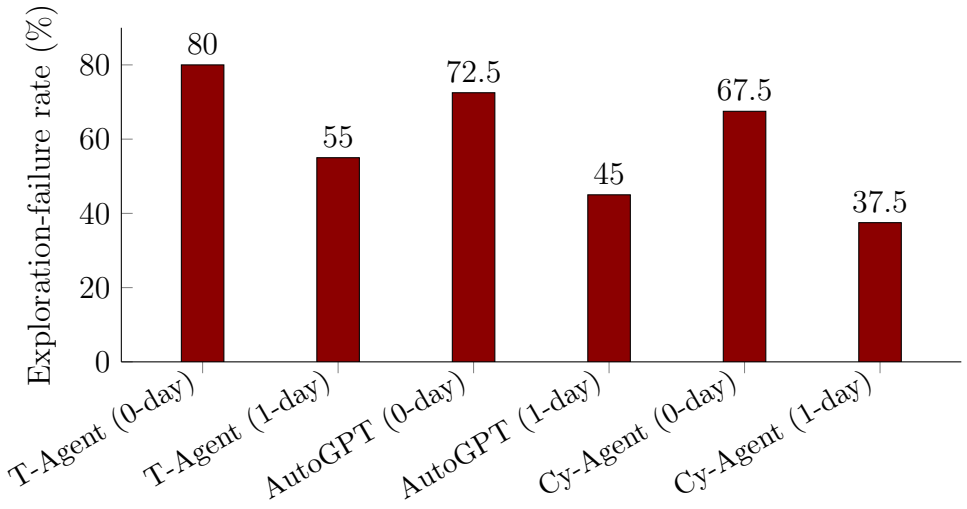

FIGURE 1.1: Exploration-failure rate as the dominant CVE-Bench failure mode, by agent and mode (data from CVE-Bench's Table 5 failure-mode breakdown [1]).

Failure Mode 2 — The dependency-reasoning gap. PentestEval [2] decomposes the pentesting pipeline into six stages — Information Collection (IC), Weakness Gathering (WG), Weakness Filtering (WF), Attack Decision-Making (ADM), Exploit Generation (EG), and Exploit Revision (ER) — and finds the opposite bottleneck at the *planning* level: ADM, which requires reasoning about prerequisite dependencies between candidate weaknesses, is the single lowest-scoring stage (Spearman correlation 0.25 against expert judgement). PentestEval's ground-truth-injection ablation quantifies roughly how much room for improvement exists at each stage, summarized in Table 1.1.

ADM delivers the largest single-stage marginal increment (+0.14) of the three tested — measured, notably, on top of an already ground-truthed WG+WF pipeline, not

TABLE 1.1: PentestEval ground-truth-injection ablation: marginal gain from replacing each stage's output with expert ground truth, applied cumulatively on top of the SMP baseline [2].

Configuration	End-to-end success	Marginal gain
SMP (baseline)	0.31	–
+ GT Weakness Gathering (WG)	0.50	+0.19
+ GT Weakness Filtering (WF)	0.53	+0.03
+ GT Attack Decision-Making (ADM)	0.67	**+0.14**

in isolation. Any dependency structure an agent grows on its own, without ground truth, is necessarily a weaker approximation than this, so a realistic ceiling for any such approach would have to sit below 0.67 rather than above it.

The gap not yet closed in the reviewed literature. Systems that target exploration breadth (HPTSA [6], VulnBot [8], and similar team-dispatch architectures) tend to dispatch flat, undifferentiated task lists with no explicit prerequisite or dependency model. Systems that target dependency reasoning (PentestEval's SMP baseline, CHECKMATE-style classical-planning approaches [9]) undergo evaluation on curated scenarios with pre-enumerated weakness sets and do not appear to scale to open-ended, wide-surface exploration. The reviewed papers do not contain a system that combines exploration over a dependency-aware structure, prerequisite relationships grown dynamically from live discovery during a mission, and evaluation across more than one independently benchmarked attack-surface family. Investigating whether closing this compound gap is achievable, and whether doing so is actually worthwhile, is the problem this thesis sets out to explore.

1.3 Objectives

Guided by the two gaps identified in Section 1.2, the following working goals guide this thesis. These reflect the research questions under active investigation, not a checklist of work already completed. The exact scope and design of each goal will likely need revision as implementation and early experiments provide feedback that the literature review alone cannot.

1. To investigate whether representing candidate vulnerabilities as a graph with explicit prerequisite relationships — rather than as an unordered list — can meaningfully improve how an autonomous agent chooses what to explore next. This includes formalizing such a structure (referred to here as a **Vulnerability Dependency Graph, or VDG**) to a level adequate for implementation and empirical testing.

2. To examine ways of separating what an agent has *confirmed* about a target from what it merely *hypothesises*, to allow independent assessment of discovery quality and planning quality.

3. To determine an appropriate multi-layer orchestration structure for the agent — drawing on the structural patterns that recur across the strongest-performing systems in the reviewed literature — and to develop this structure to the point where implementation becomes possible.

4. To consider whether allowing an agent to keep and reuse strategies across engagements offers a genuine advantage, or whether it primarily introduces risk (for example, a strategy that succeeded against one version of a technology being harmful against a different version).

5. To identify which existing, published benchmarks suit this class of system, and to begin drafting an evaluation plan around them rather than constructing new benchmarks independently.

1.4 Scope of the Project

Based on the literature review so far, this project bounds itself by one governing rule: work proceeds only with attack surfaces for which a dedicated, reusable, oracle-backed benchmark already exists in the reviewed literature, rather than constructing new benchmarks independently. This decision keeps the evaluation grounded and reproducible. The scope may narrow further once early implementation work provides more information about what is achievable within the remaining timeline.

Table 1.2 summarises the attack-surface families currently considered in scope and the benchmarks that govern any claim made about them.

TABLE 1.2: Attack surfaces currently considered in scope, and their candidate governing benchmarks (preliminary).

Attack surface	**Candidate benchmarks**	**Representative vulnerability types**
Web application (HTTP/HTML)	Fang et al. 15-CVE sandbox [5]; HPTSA 14-CVE zero-day suite [6]; CVE-Bench (40 CVEs) [1]; PentestEval (346 tasks) [2]	SQLi, XSS, CSRF, SSRF, SSTI, LFI, file-upload RCE, IDOR, authentication bypass, JWT forgery
GraphQL APIs	PrediQL 6-API suite [10]	Schema abuse, dependency-chain injection, batched-auth bypass, IDOR
Multi-host / Active Directory	Incalmo MHBench (40 environments) [11]	Lateral movement, credential reuse, privilege escalation

What is currently left out. General REST API exploitation is not assessed directly: no standardised, reusable REST API benchmark comparable to CVE-Bench or PrediQL appears in the reviewed literature. Some underlying techniques from the REST API testing literature (dependency inference between API calls, response-feedback pruning) may still be useful internally, but REST-API-specific results are not part of the plan. Binary exploitation, physical- and network-layer attacks, and social engineering are also outside the current scope, for the same reason — the absence of a suitable existing benchmark in the reviewed literature.

1.5 Expected Contributions

The intent here is to avoid overstating what this thesis has established at this point. The authors have not yet implemented or assessed any of the following. What follows is the direction considered worth investigating, framed as working hypotheses rather than results already in hand. This framing will need revision as implementation and early experiments provide evidence the literature review alone cannot.

- **Dependency-aware exploration.** The primary hypothesis is that combining exploration-style candidate selection with an explicit, dynamically grown model of prerequisite relationships between vulnerabilities — rather than treating either concern in isolation — can improve how thoroughly and how sensibly an autonomous agent searches a target. The plan is to test this against CVE-Bench and PentestEval once an implementation exists, but a position on expected performance numbers is not yet available with confidence.

- **Reuse of prior experience across missions.** The second line of inquiry examines whether allowing an agent to keep and selectively reuse strategies from earlier engagements produces a measurable benefit on targets that share underlying technology, without introducing the negative-transfer risk described in Section 1.6. Whether this ends up as a core research question or supporting infrastructure depends on what early design work shows.

- **An evaluation grounded in existing benchmarks.** Independently of whether the above hypotheses hold, the plan intends to run the evaluation consistently across more than one existing, independently published benchmark, with standardised oracles and separated per-surface reporting — since this was not done consistently across the papers reviewed.

These further mechanisms recur across the surveyed literature — structured communication between agent components, retry-and-diagnose validation loops, and detailed logging — and appear to be necessary engineering infrastructure for any implementation of the above rather than independent research questions in themselves. The thesis will revisit this framing as implementation progresses.

1.6 Challenges

The challenges listed here surfaced repeatedly during the literature review, and they will likely shape the work over the next six months regardless of which specific

design choices are ultimately made. They appear here, at an early stage, rather than waiting to encounter them during implementation:

1. **Inferring dependencies without ground truth.** Any prerequisite structure the agent builds on its own comes from LLM inference rather than expert annotation, and this represents the single riskiest part of whatever gets built. Before relying on such a structure for any evaluation, the plan is to run a small pilot study against PentestEval's expert-annotated dependencies to get a sense of how accurate the inference actually is.

2. **Sandbox results may not transfer to the real world.** Fang et al. [4] found only 1 exploitable XSS in 50 real-world candidate sites (2%), against 73.3% in a matched sandbox. This gap is worth keeping in mind throughout, to avoid presenting sandbox numbers as if they were real-world numbers.

3. **Some benchmarks are small.** PrediQL's GraphQL suite, for instance, covers only 6 APIs — real and standardised, but far smaller than benchmarks like CVE-Bench. One must state any conclusion drawn from it as exploratory rather than definitive.

4. **Cost is a moving target.** Whatever cost-per-exploit figures are eventually reported, they reflect a specific model and pricing at a specific moment, and this requires explicit statement rather than presenting cost as a fixed property of the approach.

5. **Reused strategies could backfire.** A strategy that worked against one version of a piece of software could plausibly be actively harmful against a different version. The reviewed papers do not appear to address this risk, which is one reason the design treats it as worth attention from the start.

6. **Sensitivity to configuration choices.** If the approach depends on tunable parameters, checking how sensitive results are to those choices matters, rather than reporting numbers from a single, possibly fortunate, configuration.

7. **Honesty about what generalises.** If testing spans more than one attack surface, the thesis must distinguish between "a shared design" and "one literally unmodified system", since surface-specific components will likely prove necessary either way.

1.7 Organization

This report reflects the first stage of work on this thesis, and its organization serves as a plan for the full document the authors expect to submit in approximately six months, not a description of chapters that already exist in complete form. Chapter 2 presents a review of a focused selection of papers on autonomous VAPT, organized by architectural approach, and identifies the gaps in prior work that motivated the direction described in Sections 1.2 and 1.3; this chapter is the most developed at present. Chapter 3 presents the proposed system design as it develops — currently sketched at a high level and subject to revision as implementation proceeds. Chapter 4 lays out the work breakdown, timeline, and milestones for the remaining months, and stays current as work progresses. Chapter 5 specifies and, eventually, reports on the evaluation plan once an implementation exists to assess. Chapter 6 summarises the completed work, and that chapter takes substantive form only much later in the thesis timeline.

Chapter 2

Literature Review and Related Work

2.1 Domain Overview

This chapter reviews the recent literature on LLM-based autonomous penetration testing and identifies, at a preliminary level, the patterns and gaps that motivate the research direction described in Chapter 1. The review covers a focused selection of eleven papers, chosen as the most directly relevant to the research question from a broader set of papers read during the preparation of this report. Table 2.1 provides an overview of these eleven papers; the sections that follow discuss them in more depth.

Autonomous penetration testing — the use of LLM agents to independently discover and exploit weaknesses in a target system without step-by-step human direction — has emerged as an active research area only in the last two to three years. The domain sits at the intersection of two older fields: Vulnerability Assessment and Penetration Testing (VAPT), which traditionally relies on human operators following structured methodologies (reconnaissance, weakness identification, exploitation, and reporting), and LLM agent research, which studies how language models can plan, use tools, and act toward a goal over sequential steps. The work described

in this report, RedGrid, sits in this intersection: its focus is how an LLM-driven multi-agent system can carry out the VAPT workflow autonomously against realistic, benchmarked targets.

A useful way to frame the domain is around two capabilities that an autonomous VAPT agent must exhibit simultaneously. The first is *exploration*: the agent must search a large space of candidate weaknesses across an unfamiliar attack surface without prior knowledge of which one is exploitable. The second is *prerequisite reasoning*: many real exploits are not single actions but chains, where one weakness (for example, an information disclosure or a misconfigured endpoint) must precede a second, more consequential weakness before it becomes reachable. As the literature reviewed in this chapter shows, most existing systems prioritize one of these capabilities at the expense of the other — a pattern that motivates the closer reading in the sections that follow.

The sections that follow first survey the systems and benchmarks from this selection (Section 2.2), then draw out the recurring architectural patterns across them (Section 2.3), and finally identify, at a preliminary level, where the reviewed literature appears to leave a gap (Section 2.4).

2.2 Existing Systems

This section discusses the eleven papers selected in Table 2.1, grouped by the category of contribution they represent.

2.2.1 Early Single-Agent Approaches

The earliest work in this space used a single LLM agent operating in a Reason + Act (ReAct) loop against a target. Fang et al. [4] showed that a single GPT-4 agent with browser access could autonomously exploit simplified web vulnerabilities, establishing that the basic capability exists. A later study by the same group [5] collected a benchmark of 15 real-world one-day vulnerabilities and found that GPT-4, when

TABLE 2.1: The eleven papers selected for review, with their main focus and relevance to this research.

No.	Paper	Main Focus	Why Relevant
1	Fang et al. (2024) — LLM Agents can Autonomously Hack Websites [4]	Single-agent web exploitation	Foundational: first demonstration of autonomous web attacks; establishes sandbox vs. real-world gap
2	Fang et al. (2024) — LLM Agents can Autonomously Exploit One-day Vulnerabilities [5]	Single-agent CVE exploitation	Shows GPT-4 can exploit real CVEs; key result: exploiting ≠ discovering
3	Zhu et al. (2024) — Teams of LLM Agents can Exploit Zero-Day Vulnerabilities [6]	Hierarchical multi-agent VAPT	First zero-day autonomous exploitation; introduces hierarchical planner + specialist design
4	Deng et al. (2024) — PentestGPT [7]	Single-agent with reasoning/parsing split	Identifies context-loss failure mode in long sessions; motivates structured memory and handoff
5	Kong et al. (2025) — VulnBot [8]	Multi-agent, role-specialised	Representative of team-dispatch architecture; shows flat dispatch without dependency modeling
6	Wang et al. (2025) — CHECKMATE [9]	LLM agents + classical planning	Explicit prerequisite modeling via PDDL; evaluated on curated scenarios only
7	Singer et al. (2025) — Incalmo [11]	Multi-host LLM red teaming	Extends autonomous VAPT to multi-host / Active Directory environments; declarative task API
8	Liu et al. (2025) — PrediQL [10]	GraphQL API testing with LLMs	Shows schema-based dependency reasoning for a non-web attack surface
9	Zhu et al. (2025) — CVE-Bench [1]	Benchmark for web CVE exploitation	Provides empirical evidence that insufficient exploration is the dominant failure mode
10	Yang et al. (2025) — PentestEval [2]	Stage-level benchmark	Shows attack decision-making (prerequisite reasoning) is the weakest stage in current systems
11	Wang et al. (2025) — Survey on LLM-based Autonomous Agents [3]	General LLM agent survey	Provides the conceptual and architectural vocabulary underlying all domain-specific systems

given the CVE description, exploited 87% of them — while every other tested model and every open-source scanner achieved 0%. Crucially, without the CVE description, GPT-4's success dropped to 7%, establishing an important early distinction: exploiting a vulnerability whose location the agent already knows differs fundamentally from discovering an unknown vulnerability independently. This distinction motivates much of the later work in the field.

Deng et al. [7] extended single-agent design with an explicit reasoning/generation/-parsing split aimed at reducing context loss over long testing sessions — a failure mode documented in detail there, and appearing as a concern across many later systems. PentestGPT ran on the HackTheBox and VulnHub platforms, which provide machine-level penetration testing challenges. The work identifies a fundamental tension: agents need a long conversational history to maintain awareness of what they have already tried, but long histories inflate the context window and degrade reasoning quality. This tension between memory breadth and reasoning focus is one of the recurring architectural challenges in the literature.

2.2.2 Multi-Agent and Team-Based Orchestration

A second line of work replaces the single agent with a team of cooperating agents, motivated by the observation that a single flat loop struggles to scale to wide or unfamiliar attack surfaces. Zhu et al. [6] introduced a hierarchical planner (HPTSA) that dispatches task-specific sub-agents and was the first system reported to exploit real zero-day vulnerabilities autonomously — vulnerabilities for which no CVE description is provided. This result is significant: it shows that, with the right architecture, LLM agents can discover vulnerabilities rather than merely exploit known ones.

Kong et al. [8] propose VulnBot, a comparable multi-agent framework with explicit role specialisation — separating reconnaissance, planning, and exploitation into distinct agent roles. Like HPTSA, VulnBot dispatches tasks from a central planner to specialist agents, but without explicitly modeling prerequisite relationships between

candidate vulnerabilities. The tasks go out as a flat list, and agents work through them in sequence without a formal model of what depends on what.

A general point that emerges from these multi-agent systems — and that the broader survey of the field supports [3] — is that architecture, not model scale, appears to be the dominant variable in autonomous agent performance. A well-structured pipeline tends to outperform an unstructured loop even when the latter uses a more capable base model. This architectural primacy motivates close attention to the structural design of RedGrid.

2.2.3 Planning-Augmented Approaches

A smaller set of papers pairs LLM agents with a more classical planning component, aiming to give the agent an explicit model of prerequisites between actions rather than relying purely on the LLM's implicit reasoning. Wang et al. [9] combine LLM agents with classical planning techniques (PDDL-style operators) for penetration testing, and report stronger performance on the specific scenarios where this planning applies. These systems typically require enumerating the relevant weaknesses in advance, which limits their applicability to open-ended discovery on an unfamiliar attack surface — a limitation discussed further in Section 2.4.

2.2.4 Non-Web Attack Surfaces and Benchmarks

The final group of papers covers two non-web attack surfaces and the two benchmark studies that provide the quantitative evidence for the failure modes described in Chapter 1.

Singer et al. [11] extend autonomous penetration testing to multi-host and Active Directory-style network environments, where the agent must perform lateral movement, credential reuse, and privilege escalation across a chain of networked hosts. This is a qualitatively different challenge from single-target web testing: the agent must maintain awareness of its progress across an evolving network of compromised

and uncompromised hosts. Incalmo addresses this partly through a structured, declarative task vocabulary that keeps the agent's planning language manageable. Liu et al. [10] take a different direction, targeting GraphQL APIs specifically. Their work is notable for introducing schema-based dependency reasoning within the API testing context: a GraphQL schema encodes relationships between types and fields that the agent can exploit to guide its testing strategy, drawing on the same underlying intuition about prerequisite relationships that motivates this work.

On the evaluation side, Zhu et al. [1] introduce CVE-Bench, a benchmark of 40 critical (CVSS $\geq$ 9.0) real-world web-application CVEs with oracle-backed pass/fail signals. Its failure-mode analysis provides the clearest available quantitative evidence that insufficient exploration — not reasoning quality — forms the primary bottleneck in current agents. Yang et al. [2] introduce PentestEval, which decomposes the penetration testing pipeline into discrete stages and measures agent performance at each stage. Its finding that attack decision-making — reasoning about which weakness to pursue next, given what the agent has discovered so far — ranks as the weakest stage provides complementary evidence to CVE-Bench: where CVE-Bench identifies exploration breadth as the failure, PentestEval identifies prerequisite reasoning as the failure, and together they suggest that the two capabilities need joint attention.

2.3 Comparative Analysis

Table 2.2 summarises the eleven reviewed systems and benchmarks along four dimensions that recurred throughout the literature review: whether the system uses a single agent or a coordinated team, whether it models dependencies between candidate weaknesses explicitly, whether it retains any form of memory across sessions, and the benchmarks used in its evaluation.

A few patterns are visible from this comparison. First, systems that scale to a wide or unfamiliar attack surface (HPTSA, VulnBot) dispatch tasks flatly, without an explicit model of which weaknesses depend on which others. Second, the one system

TABLE 2.2: Comparison of systems and benchmarks reviewed, across four recurring architectural dimensions.

System / Work	Architecture	Dependency Modeling	Cross-Session Memory	Benchmarks
Fang et al. [4]	Single agent (ReAct)	None	None	Simplified websites
Fang et al. [5]	Single agent (ReAct)	None (given CVE desc.)	None	15 one-day CVEs
HPTSA [6]	Hierarchical planner + task agents	None (flat dispatch)	None	15 zero-day CVEs
PentestGPT [7]	Single agent, split reasoning/parsing	Implicit (LLM only)	None	HTB/VulnHub machines
VulnBot [8]	Multi-agent, role-specialised	None (flat dispatch)	None	HTB-style scenarios
CHECKMATE [9]	Agent + classical planner	Explicit, pre-enumerated	None	Curated scenarios
Incalmo [11]	Multi-host orchestration	Partial (host/-credential graph)	None	MHBench (40 envs.)
PrediQL [10]	LLM-guided fuzzer	Schema-derived	None	6 GraphQL APIs
CVE-Bench [1]	Benchmark (no agent)	N/A	N/A	40 critical web CVEs
PentestEval [2]	Benchmark (modular pipeline)	Explicit (ground-truth injected)	None	12 real-world scenarios
Wang et al. [3]	Survey	N/A	N/A	General (survey)

that does model dependencies explicitly (CHECKMATE) undergoes evaluation on curated scenarios where relevant weaknesses are pre-enumerated, and it's not clear from the reviewed paper how well this approach extends to open-ended discovery. Third, cross-session memory — the ability for an agent to keep and reuse what it learned in a previous engagement against a similar target — is absent from every system in the table. Finally, the two benchmark papers (CVE-Bench and PentestEval) each provide stage-level failure analysis that points in the same direction: existing systems are weakest precisely at the combination of broad exploration and structured prerequisite reasoning that handling a wide, unfamiliar attack surface

demands.

These observations are preliminary, and the following section discusses their implications for the research direction this thesis pursues.

2.4 Research Gap

The literature reviewed in this chapter suggests, tentatively, that autonomous VAPT systems have so far addressed one of two capabilities at a time, rather than both simultaneously. Table 2.3 summarises this observation.

TABLE 2.3: Observed split in the reviewed literature between exploration-oriented and dependency-reasoning-oriented systems (preliminary reading).

Area	**What the reviewed literature shows**	**Observed limitation / open area**
Exploration breadth	HPTSA and VulnBot scale to wide, unfamiliar attack surfaces using hierarchical team dispatch	Neither models prerequisite relationships between discovered weaknesses; both dispatch tasks as flat lists
Prerequisite / dependency reasoning	CHECKMATE introduces classical planning with explicit prerequisites; PentestEval shows that ground-truth attack decision-making (ADM) produces the largest single-stage improvement (+0.14)	Both require weaknesses to be pre-enumerated; applicability to open-ended discovery is unclear from the reviewed papers
Cross-session memory	No system in the reviewed set retains and reuses strategies across back-to-back missions	An open area that may warrant investigation, though its value relative to the above two concerns remains unclear
Cross-surface evaluation	Every reviewed system undergoes evaluation against a single attack-surface family	No system evaluates the same architecture across web, GraphQL, and multi-host surfaces simultaneously

On one side, the reviewed literature shows that systems built for broad exploration of an unfamiliar attack surface — HPTSA, VulnBot, and similar team-dispatch architectures — do not appear to model prerequisite relationships between candidate weaknesses explicitly. They search widely, but without a formal notion of which weaknesses must precede others. On the other side, systems that do reason

explicitly about such dependencies, including CHECKMATE's classical-planning layer and PentestEval's ground-truth attack decision-making configuration, require scenarios that list the relevant weaknesses in advance. How well these approaches scale to open-ended discovery on a wide, unfamiliar surface remains unclear from the reviewed papers.

This split appears in how the two most detailed benchmark studies in this selection report their results. CVE-Bench's own failure analysis attributes the majority of unsuccessful attempts on its hardest real-world CVEs to insufficient exploration rather than to reasoning quality. PentestEval's stage-level breakdown finds that the attack-decision-making stage — reasoning about which weakness to pursue next given what the agent has already discovered — consistently ranks as the weakest stage across the systems it evaluates. Taken together, these two findings point, at a preliminary stage of the review, at two sides of what may be the same underlying gap: the reviewed literature does not yet appear to contain a system that combines broad, open-ended exploration with an explicit, dynamically constructed model of dependencies between discovered weaknesses.

To be clear, this reflects an early stage reading of the literature. The gap described above serves as a working hypothesis to guide the rest of the thesis, rather than as a finalized problem statement. The architecture and approach of RedGrid will develop and receive justification more carefully as the work progresses.

2.5 Summary

This chapter has reviewed eleven papers directly relevant to autonomous, LLM-driven penetration testing, covering early single-agent systems, multi-agent orchestration frameworks, planning-augmented approaches, non-web attack surfaces, and the benchmark studies that provide the quantitative evidence for the field's current limitations. A comparative reading of this literature points, tentatively, to a recurring split between systems optimised for broad exploration and systems optimised

for dependency-aware reasoning, with the reviewed literature not appearing to contain a system that addresses both simultaneously, and with few systems assessed across more than one attack-surface family.

As this report reflects an early stage of the thesis, the survey above serves as a snapshot of work completed so far rather than a closed literature review. Over the coming months, the review deepens — in particular around the planning-augmented systems and the question of cross-session memory — and the observations in Section 2.4 motivate a more precisely scoped research question and approach as the thesis develops.

Chapter 3

Proposed Methodologies

3.1 System Overview

3.2 System Requirements

3.3 Proposed Architecture

3.4 System Workflow

3.5 Tools and Technologies

Chapter 4

Execution Plan

4.1 Work Breakdown Structure

4.2 Project Timeline

4.3 Milestones

4.4 Resource Requirements

4.5 Risk Assessment

Chapter 5

Experimental Results

5.1 Evaluation Plan

Chapter 6

Conclusions

6.1 Summary

6.2 Expected Deliverables

6.3 Future Works

Bibliography

[1] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A benchmark for AI agents' ability to exploit real-world web application vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025.

[2] Ruozhao Yang, Mingfei Cheng, Gelei Deng, Tianwei Zhang, Junjie Wang, and Xiaofei Xie. PentestEval: Benchmarking LLM-based penetration testing with modular and stage-level design. Preprint, 2025.

[3] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhi-Yuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2025. arXiv:2308.11432.

[4] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. LLM agents can autonomously hack websites. arXiv preprint arXiv:2402.06664, 2024.

[5] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM agents can autonomously exploit one-day vulnerabilities. arXiv preprint arXiv:2404.08144, 2024. v2, cs.CR.

[6] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. Teams of LLM agents can exploit zero-day vulnerabilities. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026. arXiv:2406.01637.

[7] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, Philadelphia, PA, 2024. USENIX Association.

[8] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. VulnBot: Autonomous penetration testing for a multi-agent collaborative framework. arXiv preprint arXiv:2501.13411, 2025.

[9] Lingzhi Wang, Xinyi Shi, Ziyu Li, Yi Jiang, Shiyu Tan, Yuhao Jiang, Junjie Cheng, Wenyuan Chen, Xiangmin Shen, Zhenyuan Li, and Yan Chen. Automated penetration testing with LLM agents and classical planning. arXiv preprint arXiv:2512.11143, 2025.

[10] Shaolun Liu, Sina Marefat, Omar Tsai, Yu Chen, Zecheng Deng, Jia Wang, and Mohammad A. Tayebi. PrediQL: Automated testing of GraphQL APIs with LLMs. arXiv preprint arXiv:2510.10407, 2025.

[11] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. Incalmo: An autonomous LLM-assisted system for red teaming multi-host networks. arXiv preprint arXiv:2501.16466, 2025.